

元 n ：通过涌现深度实现递归自我改进

Zae Myung Kim¹ Young-Jun Lee¹ Seungyeon Jwa² Dongyeop Kang¹

¹University of Minnesota ²Seoul National University

{kim01756, lee05727, dongyeop}@umn.edu amyj97@snu.ac.kr

摘要

自我改进的智能体 (LLM agents) 优化的是答案，而非产生答案的过程。引入元层次的系统将该层次固定不变，而自我编辑的系统必须保留部分编辑机制不动以维持稳定，从而将其实现的元深度限制在大约两层。本文提出元 n ，它保持元操作不变，转而对输入进行递归。该操作 Ω 被反复应用于其自身的产物，读取下层求解器栈的轨迹以及生成这些轨迹的代码，然后将下一层编写为策略性预处理步骤和可调用辅助函数库。由于 Ω 从不改变，它不会破坏系统稳定性；又因其输入严格增长，每一层都能从比上一层更高的视角进行推理。深度由收敛决定，而非预先固定，并且进化档案在层链上进行搜索。在两种骨干模型上，元 n 在所有八个基准测试系列中均优于先前的自我改进智能体。最突出的案例是 ARC-AGI-2，该基准旨在抵抗技能记忆，而只有它取得了非零分数。消融实验表明，递归带来的大部分收益来自每一层传递给下一层的条件信息，并且随着深度增加，不同的层角色自然涌现，尽管没有任何提示预先规定这些角色。代码可在 <https://github.com/minnesotanlp/meta-n> 获取。

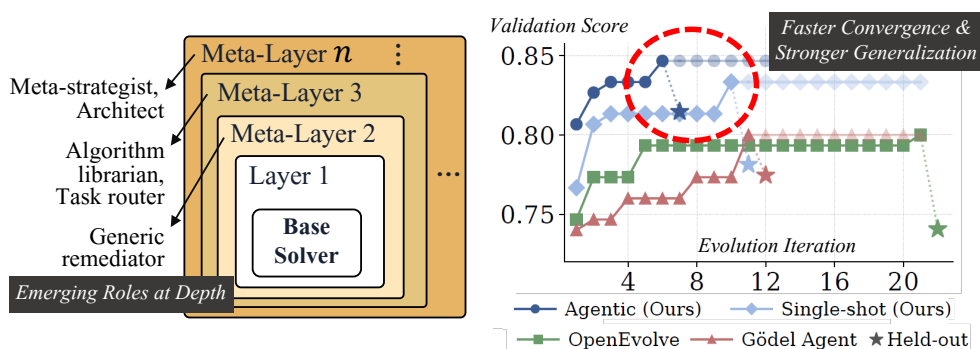


图 1：元操作 (Meta) n 概览。 (左) 单一通用元操作 Ω 被递归应用，每个元层包裹其下方的堆栈 (基础求解器 $\rightarrow$ 第 1 层 $\rightarrow$ 元层 2, 3, ..., n)。角色随深度自然涌现，无需人工设计。(右) 在 LawBench 罪名预测任务上的搜索进展，两种元操作 n 变体均比哥德尔智能体 (Gödel Agent) [尹等人, 2024年] 和开放进化 (OpenEvolve) [夏尔马, 2025年] 攀升更快且最终水平更高。

1 引言

智能的一个关键组成部分是元推理能力，即跳出自身推理链条、获得超越该链条的洞见的能力 [Russell 和 Wefald, 1991, Kounios 和 Beeman, 2014, Ackerman 和 Thompson, 2017]。但一个系统如何能跳出自身？在《哥德尔、埃舍尔、巴赫》[霍夫斯塔特, 1979] 中，霍夫斯塔特认为智能和自我意识并非源于任何单一规则，而是源于奇异循环，即自指过程——系统自身的规则被递归应用，回折并作用于其自身产物。当今的自改进大语言模型智能体恰恰未能做到这一点。它们为 NP 难优化问题编写算法、操作终端环境、提出数学构造，然而面对失败的主流应对方式是自我精炼 [马丹 (Madaan) 等人, 2023; 辛恩 (Shinn) 等人, 2023] (检查、修改、重试)，这是一种单层反思，其中同一机制负责生成、诊断和修复。现有智能体改进的是答案，而非改进答案的过程。

近期工作引入了显式的元层级：进化方法维护候选程序种群 [Romera-Paredes 等人, 2024; Novikov 等人, 2025; Sharma, 2025]; 自修改智能体重写自身源代码的部分内容 [Yin 等人, 2024; Zhang 等人, 2025a, 2026; Zelikman 等人, 2024]; 元脚手架系统在基础模型周围的基础设施上进行搜索 [Hu 等人, 2025; Zhang 等人, 2025b; Lee 等人, 2026; Fernando 等人, 2024; Agrawal 等人, 2026; Liu 等人, 2026]。

但元机制本身是固定的。搜索循环、变异算子和选择规则不会被修改，而每个确实进行自我编辑的系统都必须保持某个驱动层固定以确保稳定性，从而将实际达到的元深度限制在 ~ 2.5 。我们在第 2.1 节中使这些术语精确化 (图 2; 完整比较见附录 B)。

我们围绕这一原则构建元 n 。并非重写改进器本身，而是固定一个通用的元操作 Ω ，并反复将其应用于自身输出，“给定下方系统的完整执行上下文，编写改进它的代码。”该循环通过 Ω 自身累积的产物进行折叠。由于 Ω 将自身先前的输出作为输入，每一层都能比上一层看到更多其下的系统，从表面缺陷到策略选择，再到元策略推理 (图 1)。作为隔离检验，从元 n 中移除递归会使 CO-Bench 档案库最佳验证从 0.845 降至 0.714，仅递归一项就带来 +0.131 的增益。该效应在第二个骨干网络和第二个基准上重现，且在内循环留有余地最大的地方最为显著 (§ 3)。

一个具体示例。在 CO-Bench 的 `assignment_problem` 上，单次大语言模型调用生成一个导入 `scipy.optimize` 的脚本；沙箱缺少 `scipy`，脚本崩溃，任务得分为 0.0。扁平自精化循环重试基于相同导入的解决方案的变体。元 n 的深度 2Ω 同时查看全部 36 个任务，观察到多个任务因类似的依赖错误而失败，并决定生成一个预处理步骤和一个代码库辅助函数：“不要使用 `scipy`；这里有一个 `validate_output()` 实用工具。”深度 2 求解器重新实现了匈牙利算法，得分为 1.0。深度 3 随后看到新的迹以及产生这些迹的深度 2 代码，并决定是保留、精化还是覆盖。扁平循环仅看到迹，无法执行此类推理。 Ω 的重复应用构建了一个栈，其中每一层既是代码的生产者（指导和库函数），也是下一次 Ω 调用的对象。我们不预先指定层深度，而是让其增长，直到 Ω 不再发现改进。更深的层学习何时应用、抑制或组合原语，因为每次 Ω 调用都会看到先前层的代码及其表现。

在涵盖组合优化、文本分类（症状到疾病和法考基准）、终端智能体任务、数学发现（阿尔法进化数学和抽象推理语料库-通用人工智能-2）、符号回归以及算法加速的八个基准测试中，元 n 显著提升了单次调用求解器的性能，且无需任务内迭代。相较于以往的自改进智能体，其最大优势体现在最困难的基准测试上。例如，在抽象推理语料库-通用人工智能-2 [乔莱等人, 2026] 的留出集上，元 n 是唯一能够解决任何任务的系统，而开放进化和哥德尔智能体均未能解决任何任务。

本文的贡献。 (1) 元 n 框架 (第 2 节)：一种递归架构，其中单一通用元操作 Ω 被反复应用，构建一个层级化智能体栈，其深度由收敛性决定。据我们所知，这是首次证明超过两层的元深度会产生结构上不同的层级，而非冗余层级。 (2) 进化编排 (第 2.4 节)：对层链进行多候选档案搜索。 (3) 实证证据 (第 3 节)：在八个基准和两个骨干网络上，元 n 在每个基准族上都优于先前的自改进智能体，在最难的留出任务上优势最大。

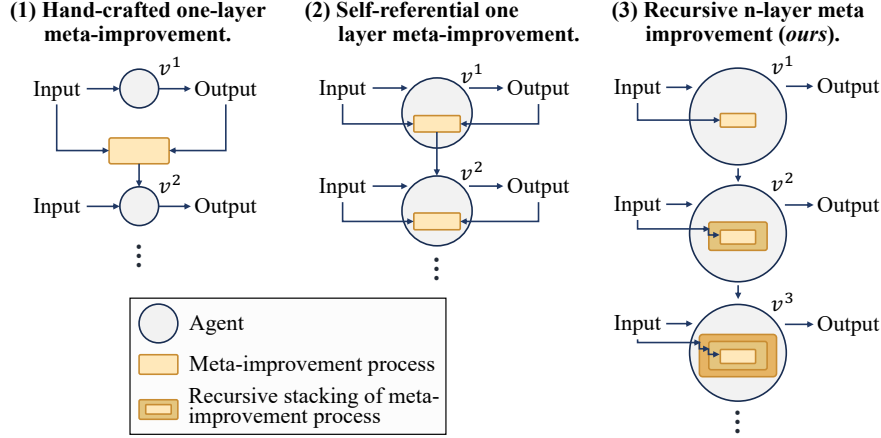


图 2：元改进的三种范式。圆圈表示智能体（连续版本 $v^1, v^2, \dots$ ）；琥珀色方框表示元改进过程，嵌套方框表示递归堆叠的元过程。（1）手工设计的一层元（例如 FunSearch、AlphaEvolve、ADAS）：固定的外部元过程重写智能体，但元机制本身从不改变。（2）自指一层元（例如 Gödel Agent、DGM）：智能体编辑自己的源代码，但驱动层保持固定，将实现的元深度限制在 ~ 2.5 。（3）递归 n 层元（元 ^{n} ，本文方法）：单一元操作 Ω 在每个深度都被应用并累积，因此版本 v^d 携带 d 个嵌套元过程；深度由收敛性决定，每个新增层观察到的轨迹景观比其下一层更广阔。

2 Meta ^{n} : Evolutionary Meta-Recursion

概述。本文从单一固定元操作 Ω 构建元 n ，该操作递归地应用于其自身先前的输出。每次应用都会检查下方求解器栈在整个任务集上的表现，然后编写下一层的代码——一个简短的 Python 预处理程序，在每个任务前注入策略性上下文，以及一个求解器可以调用的小型可复用辅助函数库。下一次应用会同时看到新的性能轨迹和产生这些轨迹的代码，因此它可以判断上一层的思想是否良好，而不仅仅是答案是否错误。栈自行加深，并在没有进展时停止。第 2.1 节将现有的元改进方法组织为范式，并将本设计定位为其中一种新范式；第 2.2 节定义单个元 n 层；第 2.3 节分析递归相对于单层所获得的增益；第 2.4 节介绍增长栈的编排器。

问题表述。基准提供 N 个任务 $\mathcal{T} = \{t_1, \dots, t_N\}$ 以及一个评估器，该评估器对任务上的任何候选脚本进行评分， $\text{eval}(t_i, s) \in [0, 1]$ （在 LawBench 上，其中一个任务是罪名预测，通过 F1 评分）。每次求解器运行都会留下执行迹 τ （运行的脚本、其标准输出/标准错误、退出代码、分数以及任何评估器反馈）， Ω 可以稍后检查以诊断发生了什么。基础求解器 S_1 是任何一次性或智能体 LLM 过程，它将任务 t_i 映射到其迹 $\tau_i^{(1)}$ ；在我们的实验中 S_1 要么是单次 LLM 调用，要么是 8 轮观察/行动循环，但 Meta n 对此选择不敏感。我们的目标是构建一个求解器栈 $\{S_d\}_{d=2}^n$ ，每个求解器包装其下方的一个，以最大化平均分数 $\bar{S} = \frac{1}{N} \sum_i \text{score}(\tau_i^{(d)})$ 。深度 n 并非预先固定；它会持续增长，直到改进停止（符号表见附录表 4）。

2.1 元改进范式

为了使“自我改进的深度”在不同系统之间具有可比性，我们固定三个术语。相对于 0 级求解器，1 级过程修改求解器，2 级过程修改 1 级过程，依此类推。驱动器是控制此类修改但在运行期间自身从不被修改的组件。系统的实际元深度则是其行为在运行过程中实际发生变化的最高级别，而非其架构名义上允许的深度。根据这些术语，先前的自我改进系统分为两种范式，而 Meta n 构成第三种（图 2）。

(1) 手工构建的元系统 在求解器外部放置一个固定的搜索或进化循环。进化程序搜索维护候选程序种群 [罗梅拉-帕雷德斯等人, 2024, 诺维科夫等人, 2025, 夏尔马, 2025]; 元脚手架系统在基础模型周围搜索提示、工作流或智能体设计 [胡等人, 2025, 张等人, 2025b, 李等人, 2026, 费尔南多等人, 2024, 阿格拉瓦尔等人, 2026]。这些系统之所以稳定且可审计, 正是因为元过程是冻结的; 搜索循环、变异算子和选择规则从不修改。求解器在改进; 改进器本身不改进。实现的元深度为 1。

(2) 自指智能体 通过让智能体编辑自身源代码来内化元过程 [尹等人, 2024, 张等人, 2025a, 泽利克曼等人, 2024, 张等人, 2026]。原则上这允许无限深度; 实际上每个此类系统都固定某个驱动程序以防止自我损坏: 哥德尔智能体的动作应用程序编程接口、深度生成模型的存档维护和父代选择、超智能体的外部评估循环。因此, 可编辑表面是改进机制的严格子集, 将实现的元深度限制在 ~ 2.5 。智能体修改求解器 (第 1 层), 并且由于修改逻辑位于被编辑的源代码内部, 自我编辑也会改变未来自我编辑的进行方式 (第 2 层)。但由于冻结的驱动程序位于每个更高层内部, 第 2 层以上的任何内容都不会完全改变; 因此半学分 (即 0.5) 记录了这种部分可修改性 (各系统的详细信息见附录 B)。

两种范式共同揭示了一个困境。递归改进器以稳定性为代价换取深度, 并且 (据我们所知) 所有现存系统都通过冻结驱动程序来解决这一张力, 而这恰恰限制了递归本应带来的深度。

(3) 递归 n 层元 (元 ^{n})。元 ^{n} 消解了这一困境, 而不是沿着它进行权衡。改进器按设计被冻结, 是一个固定的由大语言模型提示的操作 Ω , 而递归被应用于其输入。由于 Ω 消耗自身累积的产物 (下方堆栈的迹以及产生这些迹的代码堆栈), 每次应用都作用于比上一次严格更高阶的输入。操作本身从不改变, 因此不会破坏系统稳定性。我们不预先固定深度, 而是让它增长, 直到 Ω 停止发现改进 (§ 2.3)。

2.2 深度 d : Ω 和 M_d 处的元层

深度 d 处的单个元层分两个阶段展开: 构建步骤写入该层 (离线执行一次), 运行步骤执行该层 (在线按任务执行)。图 3 通过 LawBench 罪名预测运行 (将刑事案件描述映射到官方罪名标签) 追踪这两个阶段, 这是本节其余部分的贯穿示例。图 3a 显示 Ω 写入第 2 至第 4 层; 图 3b 显示完成的深度 4 堆栈执行任务。构建步骤调用 Ω 检查前一深度的迹并生成新代码 C_d ; 运行步骤使用包装器 M_d 将 C_d 插入现有求解器周围, 并通过生成的堆栈运行任务。

Build-step (writing the layer): the driver Ω . Ω is a fixed LLM-prompted procedure, i.e., one prompt template that does not change between depths or benchmarks. Given the previous depth's traces $\{\tau_i^{(d-1)}\}_{i=1}^N$, the stack of code earlier rounds emitted $[C_2, \dots, C_{d-1}]$, the task descriptions $\mathcal{T}$, and the current depth d , Ω writes a new piece of code:

$$\Omega : (\{\tau_i^{(d-1)}\}_{i=1}^N, [C_2, \dots, C_{d-1}], \mathcal{T}, d) \mapsto C_d. \quad (1)$$

输出 $C_d = (f_{\text{pre}}^{(d)}, \mathcal{L}^{(d)})$ 是一个对。预处理 $f_{\text{pre}}^{(d)}$ 是一个小型的 Python 函数, 在每次求解器调用之前运行, 并注入策略上下文。在图 3a 中, 深度 3 $f_{\text{pre}}^{(3)}$ 要求对每个可能的指控进行“详尽的法律分析”, 而深度 4 版本则告诉求解器用 `reconcile_labels` 清理其标签。代码库 $\mathcal{L}^{(d)}$ 保存求解器可以调用的可重用 Python 函数, 例如在深度 2 发出的 `fuzzy_match_label()` 和在深度 3 添加的 `reconcile_labels()`。具体来说, Ω 的响应包含一个理由块、一个 `pre_process` 块, 以及零个或多个 `solver_lib:<name>` 块, 每个库函数一个 (完整的深度 2 示例见附录 F)。

保持 Ω 固定使得第 2.1 节的设计原则可操作, 因为深度 $d > 2$ 处的任何增益都必须来自更丰富的输入, 而不是改进器的改变。唯一随深度变化的部分是 Ω 摄入的迹摘要。其格式取决于深度, 从深度 3 开始, 它将注意力从单个错误转移到整个堆栈的模式 (详情见附录 E.2)。这种格式部分地支撑了第 3 节中分析的战术与战略注意力分工, 但它只塑造了每一层看到的内容; 没有提示规定某一层编写的代码。

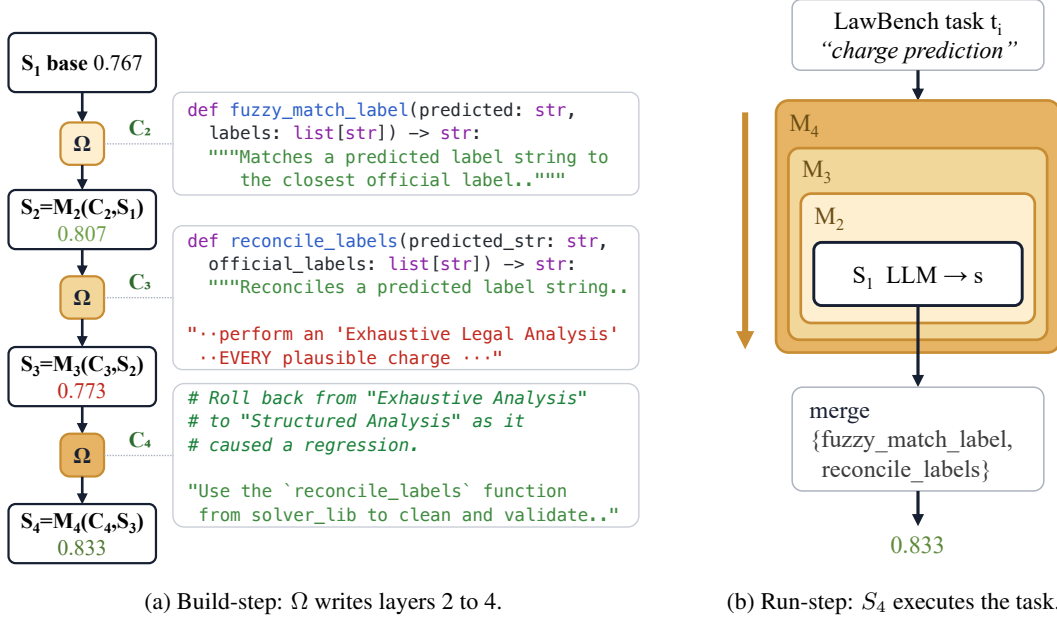


图 3：元层的两个阶段，通过 LawBench 指控预测运行（任务得分 $\in [0, 1]$ ）进行追踪。 (a) 构建步骤：每次 Ω 调用读取下方堆栈的迹，从深度 3 开始也读取其代码，然后编写下一层的代码 C_d 。深度 2 发出辅助函数 `fuzzy_match_label`；深度 3 添加 `reconcile_labels`，但将其与过度规定的“详尽法律分析”指令耦合，导致得分倒退；深度 4 在保留辅助函数的同时撤销了该指令。(b) 运行步骤：完成的 $S_4 = M_4 \circ M_3 \circ M_2 \circ S_1$ 将任务向内传递给基础求解器，并将累积的辅助函数合并到最终脚本中。

运行步骤（执行该层）：包装器 M_d 。 当任务 t_i 在深度 d 进入时，包装器 M_d 围绕先前的求解器 S_{d-1} 分五个阶段嵌入 C_d ，如图 3b 所示。在进入过程中，(1) 最外层 $f_{\text{pre}}^{(d)}$ 首先运行，为该任务生成策略上下文字符串 ctx_d ，(2) ctx_d 向内穿过下方各层的预处理过程，每一层对其进行细化。在返回过程中，(3) 基础求解器看到合并后的上下文并返回脚本 s ，(4) 联合代码库 $\mathcal{L}^{(2)} \cup \dots \cup \mathcal{L}^{(d)}$ 被前置到 s ，深层在名称冲突时覆盖浅层，(5) 结果在沙箱中执行以产生深度为 d 的迹 $\tau_i^{(d)}$ 。由于 M_d 不触及内部求解器的内部结构，包装器可以干净地组合，因此深度为 n 的求解器只是嵌套调用，最终在基础求解器 S_1 处结束：

$$S_d = M_d(C_d, S_{d-1}) = M_d \circ M_{d-1} \circ \dots \circ M_2 \circ S_1. \quad (2)$$

M_d 从不修改内部求解器、任务对象或先前层的库，并且 M_d 在将注入的代码前置之前，在隔离的命名空间中静态验证并冒烟测试该代码（理由与安全措施见附录 D）。

2.3 组合递归：策略、战术与 Ω 所见

递归相对于扁平改进器带来两点优势：策略/战术空间的乘法覆盖，以及为改进器本身提供严格更丰富的输入。第一点关乎 d 层堆栈在运行时能够表达的行为；第二点关乎 Ω 在编写下一层时必须审视的内容。

策略制约战术。 在运行步骤中，每个预处理过程接收上一层的上下文字符串，并为下一层发出自己的上下文字符串， $f_{\text{pre}}^{(d-1)}(t_i, \text{ctx}_d) \rightarrow \text{ctx}_{d-1}$ 。因此各层相互制约；一层发出的上下文受到从上方传递下来的上下文的影响，所以较深的层设定框架，较浅的层填充细节。在深度 d 处有 k_d 种不同行为时，制约机制允许最多 $\prod_{d=2}^n k_d$ 种联合配置，而具有相同行为的无制约扁平架构仅有 $\sum_d k_d$ 种（在 $n=4$ 和 $k_d=3$ 处，27 对 9）。这是一个

可表达配置的上界，而非实测计数。其可测量的特征在第 3 节中表现为存档最佳与最佳单链得分之间的差距，以及一项将大部分递归增益归因于该制约通道的消融实验。图 3b 展示了制约机制的作用。最外层首先运行并设定策略框架，每个内层在该框架内细化指导。基础求解器随后接收所有层的合并上下文以及联合库。

条件化也意味着各层之间可能相互干扰。更深层的指令可能覆盖有用的浅层指导，我们在实践中观察到此类逐任务回归（在 § 3 中量化）。运行示例正是这样一个循环。在图 3a 中，深度 3 的指令将得分从 0.807 回退到 0.773，尽管随附的辅助器是可靠的。由于深度 4 的 Ω 同时读取代码栈 $[C_2, C_3]$ 和迹（公式 1），它能够将回归归因于指令而非辅助器。其理由明确陈述了回退（“从穷尽分析回退到结构化分析，因为它导致了回归”），其 C_4 恢复了先前的策略，同时保留 `reconcile_labels`，恢复到 0.833。因此，干扰是表达力的代价而非缺陷，并且在两个层面上得到修复：在链内由 Ω 自身修复（如此处所示），在链间由 § 2.4 的存档与整合机制修复。

Ω 所见与扁平自精炼所见。扁平自精炼循环在第 j 次迭代时仅观察其前 $j-1$ 次迭代的迹，这是一个累积的“发生了什么”日志，从不包含导致其发生的代码。根据构造，深度 $d \geq 3$ 处的 Ω 同时看到前一度度的迹和产生这些迹的代码栈 $[C_2, \dots, C_{d-1}]$ 。这使得扁平循环无论运行多少次迭代都无法执行的高阶推理成为可能，例如图 3a 中的回退（“ C_3 的指令过度约束了分析；在深度 4 将其回退”。每当 C_{d-1} 产生可观察的变化时， Ω 在深度 d 处的信息严格超集于其在深度 $d-1$ 处的信息。§ 2.1 中的固定驱动系统介于两者之间。它们的改进器仅看到其可编辑层产生的输出，这是 $\text{Meta } n$ 的 Ω 所见内容的严格子集。因此， $\text{Meta } n$ 重新定义了设计问题。收益来自于给 Ω 更多可读内容，而非重写 Ω 本身。§ 3 通过实验隔离了这一主张；在保持编排不变的情况下移除递归，在我们消融的每个骨干和基准上都降低了得分，且基础求解器越弱，下降幅度越大。

2.4 编排：线性与进化模式

两个编排器在 Ω 和 M_d 之上增长栈：一个简单的线性加深器和一个进化存档搜索。

线性元递归。最简单的编排器逐层贪婪地加深堆栈。从基础求解器 S_1 开始，每一步 Ω 观察当前迹并生成新的 C_d ，包装器 M_d 将其组合到 S_{d-1} 上形成 S_d 。循环在以下情况停止：(a) Ω 返回空代码，(b) F 个连续层未能将平均分数提高超过尺度感知容差 $\epsilon \cdot R$ (R （运行开始时冻结的分数范围），或 (c) 达到最大深度 D 。因此深度由收敛决定，而非预先规定；实际上报告的运行均未达到上限。

进化档案。线性递归对一次不幸的 Ω 调用很脆弱；一次糟糕的注入会终止链条，即使在同一深度的不同注入本可解锁进一步进展。进化变体（算法 1，附录 E.1）通过维护一个单调增长的候选链档案 $\mathcal{A}$ 并在其上搜索而非贪婪地扩展单条链来对冲风险。每次迭代采样 B 个父代，权重为 $w(c) \propto \bar{S}(c) + \alpha/(1 + \dots)$ ，子代 (c)，因此高分被优先考虑， α 项作为探索奖励，偏向扩展次数较少的链。每个父代产生 K 个子代， Ω 的采样温度循环变化以增加多样性；在父代表现不如档案的任何任务上，跨候选启发将最佳竞争迹附加到 Ω 的提示中。档案还跟踪每条链在每个任务上取得的最佳分数；这些逐任务最佳分数的均值即为档案最佳 $\bar{S}^*$ ，与最佳单链分数一同报告在每个结果表中。两者之间的差距正是档案通过让不同任务由不同链获胜所获得的收益。循环在 F 个连续无改进迭代后终止。

巩固保护。由于更深的层可能使个别任务退化（§ 2.3），编排器还支持一种巩固模式，其中每个候选针对单个焦点任务，同时继承档案为所有其他任务冻结的最佳迹，从而使逐任务最佳轨迹单调递增。

我们主张此模式因其零回归保证而非均值提升（研究及计算匹配对照见附录 G）。

智能体求解器。两种编排器均可封装单次求解的层一求解器，或封装智能体式求解器，后者运行生成、执行、观察、精炼的循环，最多 T 轮（ $T=8$ ）；智能体式求解器接收相同的预处理和代码库，并返回得分最高的脚本。在相同的进化超参数下比较两者，使我们能够分离任务内迭代的贡献与 Ω 的跨任务信号的贡献。

设计决策。我们在附录 D 中记录了其余选择（所有层使用相同模型；包装而非补丁；3:1 的失败偏向迹比率；每个基准的 ϵ 、耐心和束设置）。

3 实验

实验设置。我们在两种骨干模型上报告结果：Gemma 4 31B-IT 与 GPT-5.2；除非另有说明，每个数值均为三个随机种子（42、43、44）下的均值 $\pm$ 标准差。八个基准族涵盖三种求解器基底：Python 源代码（CO-Bench [孙等人, 2026年]，36 个 NP 难问题；AlphaEvolve Math [诺维科夫等人, 2025年]；符号回归（SR），4 个领域；AlgoTune，8 个任务；ARC-AGI-2 [肖莱等人, 2026年]，120 个任务）、Docker 沙箱内的 bash（TerminalBench 2.0 (TB2)，13 个类别共 89 个困难任务），以及针对固定下游模型的提示重写（Symptom2Disease (S2D) 与 LawBench 罪名预测）。两种 Meta n 变体在相同的进化编排下运行，使用第 2.2 节的单一 Ω 模板，分为单次模式（每个任务一次大语言模型调用；隔离 Ω 的贡献）与智能体模式（观察-行动循环，每个任务最多 8 轮）；我们按基准选择 B 、 K 、耐心值与最大迭代次数（附录 C，表 5）。我们在两处偏离了种子策略。TB2 的标准差是在 13 个任务类别上而非随机种子上计算的（GPT-5.2 下的 TB2 为单种子，s42），且没有留出划分的基准（AlphaEvolve Math、AlgoTune、SR）直接报告基准分数。完整配置与成本见附录 C。

基线与估计器。哥德尔智能体（Gödel Agent, GA）[Yin 等, 2024] 与开放进化（OpenEvolve, OE）[Novikov 等, 2025; Sharma, 2025] 是此前的自改进对比方法，在表 1 和表 2 所标注的基准上以相同基座模型运行。我们以修正后的逐任务配置报告 GA，因为其已发布的单求解器接口在 CO-Bench 上出现结构性崩溃；第 3.2 节对两者均进行了分析。OE 是一种按工件进化的对比方法（岛式/映射精英），在 CO-Bench 上以几乎相同的令牌预算运行；第 3.2 节在默认预算不同的情况下显式均衡了计算量。对于元 n ，我们在所有位置报告两种估计量：第 2.4 节的存档最优（ $\bar{S}^*$ 与最佳单链。二者之差即为逐任务选择相对于固定单链（第 2.3 节）所获得的收益。

3.1 主要结果

表 1：元 n 智能体式与先前自改进智能体的对比（Gemma 4 31B-IT，种子 42/43/44 上的均值 $\pm$ 标准差）。CO-Bench、S2D 和 LawBench 为留出测试；AlphaEvolve Math、AlgoTune（加速比 $\times$ ）和 SR（四领域均值）为基准分数。ARC-AGI-2 和 SR 基线仅在 GPT-5.2 下运行（表 2）；n/r = 未运行。

Method	CO-Bench	S2D	LawBench	AE Math	AlgoTune	SR
Meta n archive-best	0.851 $\pm$ 0.014	0.733 $\pm$ 0.015	0.815 $\pm$ 0.013	0.869 $\pm$ 0.045	$\times 15.10 \pm 2.4$	5.22 $\pm$ 0.38
Meta n best chain	0.782 $\pm$ 0.016	0.743 $\pm$ 0.034	0.796 $\pm$ 0.046	n/r	n/r	4.68 $\pm$ 0.45
OpenEvolve [Sharma, 2025]	0.814 $\pm$ 0.022	0.718 $\pm$ 0.022	0.745 $\pm$ 0.034	0.802 $\pm$ 0.052	$\times 10.45 \pm 1.8$	n/r
Gödel Agent [Yin et al., 2024]	0.451 $\pm$ 0.023	0.710 $\pm$ 0.034	0.775 $\pm$ 0.023	0.581 $\pm$ 0.061	$\times 13.22 \pm 2.7$	n/r

要点。元 n 智能体式在表 1 和表 2 的每个基准族上，至少在一个估计量上领先，且在两种骨干模型下均如此，尽管领先幅度因基准而异。在代码基板上，优势大且稳定。在 GPT-5.2 上，CO-Bench 对开放进化的领先幅度达到 $+0.168$ ，且逐种子范围不重叠；AlphaEvolve Math 以相近幅度领先；SR 在全部四个领域领先。在 ARC-AGI-2 上，差距是类别性的而非数量性的，两个基线均接近下限。在提示重写基准上，领先幅度缩小到几个百分点。元 n 在两种骨干模型下于 S2D 均值上仍领先，尽管逐种子范围重叠；LawBench 以 $+0.040$ 的优势领先哥德尔智能体。两个架构杠杆解释了这些优势：跨深度的递归以及通过代码库注入实现的跨任务迁移。

表 2：跨模型条件：GPT-5.2 骨干，种子 42/43/44 上的均值 $\pm$ 标准差。 CO-Bench 和 S2D 为留出测试；AlphaEvolve Math、SR（四领域均值）和 ARC-AGI-2（开发分数；留出测试结果在正文中报告）为基准分数。由于预算原因，LawBench 和 AlgoTune 未在 GPT-5.2 下运行；其 Gemma 列见表 1。n/r = 估计量未针对此单元格运行。

Method	CO-Bench	S2D	AE Math	SR	ARC-AGI-2
Meta ⁿ archive-best	0.870 $\pm$ 0.011	0.725 $\pm$ 0.014	0.917 $\pm$ 0.016	5.03 $\pm$ 0.20	0.331 $\pm$ 0.010
Meta ⁿ best chain	0.806 $\pm$ 0.017	0.734 $\pm$ 0.020	0.820 $\pm$ 0.024	n/r	0.123 (1 seed)
OpenEvolve [Sharma, 2025]	0.702 $\pm$ 0.025	0.721 $\pm$ 0.007	0.726 $\pm$ 0.046	3.70 $\pm$ 0.13	0.003 $\pm$ 0.001
Gödel Agent [Yin et al., 2024]	0.527 $\pm$ 0.033	0.708 $\pm$ 0.037	0.674 $\pm$ 0.068	2.45 $\pm$ 1.63	0.054 $\pm$ 0.006

ARC-AGI-2：类别性案例。ARC-AGI-2 旨在探测流体智力；其任务无法通过精炼已知技能来解决，只能通过抽象新技能来解决，因此它将两个操作层面分离开来。对象级迭代停留在底层附近，甚至 Metaⁿ 的最佳单链也仅达到 0.123，但完整的元级堆栈——其中 Ω 从轨迹中抽象出变换原语，档案库跨任务组合这些原语——达到了 0.331。在一个旨在抵抗技能记忆的基准上，增益来自元层面而非更好的对象级搜索。

CO-Bench。档案库恢复了单次大语言模型调用完全无法解决的任务。在一次代表性运行中（Gemma, s42），约束断头台切割达到 0.000 $\rightarrow$ 0.996，最大独立集达到 0.000 $\rightarrow$ 0.908。总体而言（表 1），档案库最佳在 Gemma 上领先 OpenEvolve，并在 GPT-5.2 上将差距扩大到 +0.168，且每个种子的范围互不相交；三种系统的排序在两个骨干上相同，因此差距并非骨干伪影。档案库在两个骨干上还比自身最佳单链高出 0.06 至 0.07，这是第 2.3 节乘法覆盖界的经验特征。

数学与算法发现。在 AlphaEvolve Math 上， Ω 注入了两个可调用原语 simulated_annealing() 和 basin_hopping()，将一个不可解的种子转化为强求解器；在 Gemma s42 运行中，种子从单次 0.222 提升到 0.709，从智能体 0.435 提升到 0.883。跨种子 Metaⁿ 在任一骨干下均领先两个基线，且 SR 在所有四个领域上呈现相同模式（逐领域分解见附录表 8）。AlgoTune 是唯一一个智能体变体表现不如其单次兄弟的基准（s42 上为 $\times 14.11$ vs. $\times 18.47$ ），或在两个变体都记录的七个任务上为 $\times 15.96$ vs. $\times 18.47$ （附录表 10），尽管三种子智能体均值（ $\times 15.10$ ）仍优于所有基线。该基准的预优化核函数契约已被种子提取， Ω 的额外上下文过度约束，附录 I 中的逐任务诊断使这一机制可检验。

TerminalBench 2.0。在 Docker 基板上运行任一基线都需要对其代码进行大量修改，因此 TB2 改为将 Metaⁿ 与其自身的种子进行对比。两种变体从截然不同的起点收敛到相似的绝对增益，因此 Ω 的贡献与种子强度大致正交。在 Gemma 上，单次模式将 13 类别的均值从较弱的 0.067 种子提升 +0.21，而智能体模式从 0.258 种子提升 +0.23；在 GPT-5.2 上，智能体模式达到 0.634。存档优于链的差距在此同样成立，并且该基准测试了语言自适应注入， Ω 在 13 个沙盒类别中同时生成 bash 函数和 Python 辅助脚本。

文本分类。提示重写基准限制了任何元层所能增加的上限，因为动作表面是固定标签集上的单个提示。在 S2D 上，最佳链在任一骨干下均领先两个基线，分别领先 +0.025（Gemma）和 +0.013（GPT-5.2），尽管每个种子的范围重叠，我们不声称具有显著性。LawBench 的区分度更大，存档最佳比 Gödel Agent 领先 +0.040，比 OpenEvolve 领先 +0.070。深度递进即使在此处也保持清晰。在一条代表性的 S2D 链上， Ω 从 CoT 流水线（d2）移动到 robust_label_match 辅助函数（d3），再到 classification_prompt_builder（d4）；在 LawBench 上，d2 切换到中文提示，d3 过度约束并回退，d4 将提示强度回滚，这与 § 3.3 中分析的回归-回滚循环相同。

单次模式与智能体模式。在图 4 中，线性最佳柱隔离了递归贡献，而存档最佳的残差隔离了存档本身所增加的部分。智能体模式在八个基准家族中的七个上改进了存档最佳，其中在 TB2、AlphaEvolve Math 和 SR 上的增益最大。

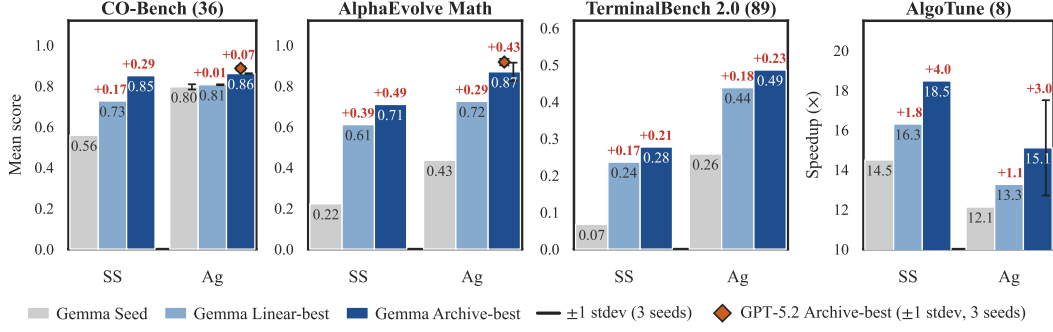


图 4：在四个代表性基准上，Meta n 变体（单次模式与智能体模式）的对比（Gemma 4 31B-IT；超参数见表 5）。SS = 单次模式，Ag = 智能体模式。每组显示种子（第 1 层求解器，在任何元层之前）、线性最佳（最佳单链）和存档最佳（跨存档的每任务最佳），并标注原始分数和相对于该组种子的增益。面板绘制验证或基准分数，保留的测试结果见表 1；AlgoTune 为加速比（ $\times$ ）。黑色须线标记三种子均值 ± 1 标准差（种子 42/43/44），橙色菱形标记三种子 GPT-5.2 存档最佳（在该骨干上运行时）。

智能体种子本身显著提升（TB2 0.067 $\rightarrow$ 0.258、AlphaEvolve 0.222 $\rightarrow$ 0.435），因此内部观察-行动循环提取出更强的第一层基线，在此基础上 Ω 进一步增加收益。智能体模式下令牌成本大约高出 4 到 10 $\times$ 倍（每个任务 8 次调用），以计算换取更丰富的迹信号；AlgoTune 是例外，其中更丰富的信号过度约束（§ 3.3）。

收敛行为。图 5 显示 Meta n 智能体在三个基准上均以更少的进化迭代达到比两个基线更高的分数，而单次变体在 S2D 上落后于两者。OE 上升但稳定在下方，GA 快速上升后在 CO-Bench 上接近 0.53 处趋于平缓，远低于 Meta n 。提高 GA 的预算并不能缩小这一差距（§ 3.2）。

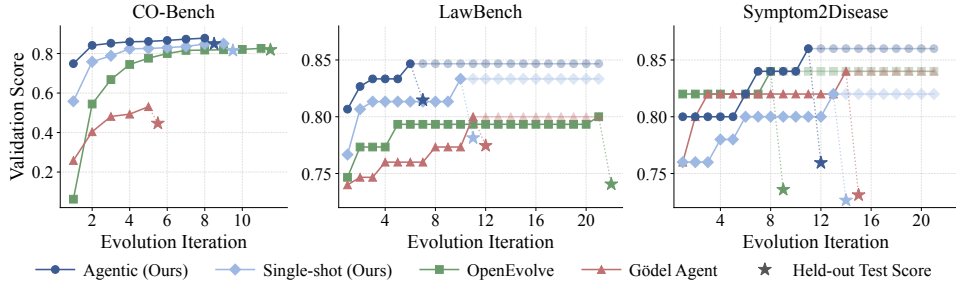


图 5：CO-Bench、LawBench 和 Symptom2Disease 上的搜索进展（Gemma 4 31B-IT，单种子运行）。线条为进化迭代过程中的最佳验证分数；星号（*）为在固定偏移处绘制的留出测试分数；淡化的标记保留早停后的最后值。

3.2 基线对等与诊断

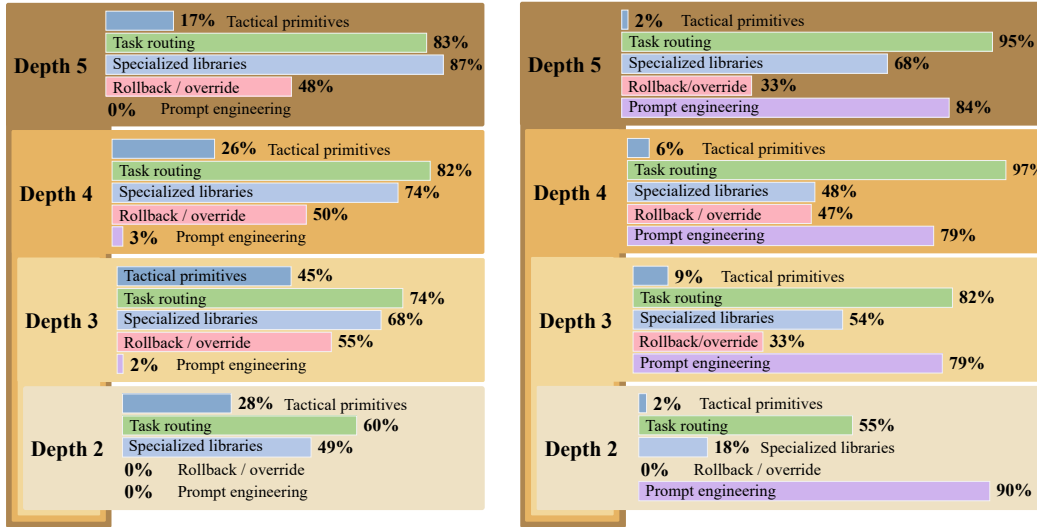
OpenEvolve：计算对等下的跨任务迁移。Meta n 与 OE 之间的架构差异在于跨任务迁移。OE 的逐工件循环无法在单次运行中跨任务共享发现的模式（例如“避免 scipy”），而 Ω 的代码库注入源自跨任务失败模式。在 CO-Bench 上，两个系统已经以几乎相同的令牌预算运行。在提示基准上，全预算 Meta n 比 OE 多花费 10 到 20 $\times$ 倍的令牌，我们将 Meta n 的总 LLM 令牌支出硬性限制为每个基准的 OE 预算，平均 485K 令牌（Gemma）。Meta n 在对等下仍然领先，S2D 0.732 $\pm$ 0.023 vs. 0.718 $\pm$ 0.022 和 LawBench 0.784 $\pm$ 0.013 vs. 0.745 $\pm$ 0.034，因此全预算差距包含计算成分，但不能归结为计算成分。Meta n 也是样本效率更高的搜索器；在 CO-Bench（Gemma）上，它使用 $\sim 13\times$ 更少的候选评估（29 对 378）优于 OE，这是运行一个跨所有任务的分组搜索而 OE 运行 36 个独立逐任务进化的结构性结果。

哥德尔智能体：两种配置，一个平台期。GA 已发布的设置要求单一求解函数覆盖全部 36 个异构 CO-Bench 任务，该函数源自一个通用数学问答求解器，根本不产生任何逐实例的 `solve()`；在两个骨干模型上均退化为 ~ 0.000 。由于这是已发布接口的结构性属性而非能力上限，我们的表格报告了修正后的逐任务配置（一个标志；相同的智能体、模型和评估器），在 Gemma 上达到 0.451 ± 0.023 ，在 GPT-5.2 上达到 0.527 ± 0.033 。即使经过修正，与 Meta n 的剩余差距仍是架构性的，而非预算性的。在 GPT-5.2 种子 42 上，将 GA 的预算从 $5\times$ 提高到 $10\times$ ，仅使留出测试从该种子默认预算的 0.502 提升到 0.615 和 0.628，且每次迭代曲线在 0.4 到 0.6 范围内振荡，没有向 Meta n 的 0.870（1700 万令牌）上升的趋势，因此我们停止在该处提高其预算。第 2.1 节的诊断在轨迹中可见。在驱动器固定的情况下，模型必须选择修改，但通常不会修改，而 Ω 总是发出注入，将“改变什么”与“是否尝试改变”分离开来（详见附录 J）。

3.3 每一深度学到了什么

涌现角色分析。尽管 Ω 的提示模板从未告诉任何层要扮演什么角色，但角色进展从累积上下文中涌现出来，我们在图 6 中保守地报告了这一点，仅统计两个独立提示的大语言模型评分者（GPT-5.2 和 Kimi-K2.6，对七个角色类别中的所有 596 个 Ω 输出进行标注）达成一致的输出。最清晰的信号是类别性的，而非渐进性的。在两个基板上，回滚在深度 2 处恰好为零，在深度 3 处出现（代码上 55%，提示上 33%），因此纠正角色是唯一一个在层有内容可纠正之前不存在的角色。渐进式转变与之并行。在代码基板上，战术原语在深度 3 处达到峰值（45%），到深度 5 时衰减至 17%，因为专用库接管了；而在提示基板上，提示工程在每个深度都饱和，战术原语从未超过 9%。

评分者间一致性平均 Cohen 的 $\kappa = 0.59$ ，在任务路由和提示工程上近乎完美，在两个最抽象的角色上最弱（0.37 和 0.30，逐类别 κ 见附录 E.3）。



(a) 代码基底（CO-Bench、AlphaEvolve Math、SR）。分母是深度 2 至 5 的 $n = 68, 125, 72, 23$ 个输出。战术原语在深度 3 处达到峰值，然后让位于专用库；提示工程全程接近零，因为没有提示可重写。

(b) 提示基底（症状到疾病、法律基准）。分母为 $n = 51, 78, 108, 63$ 。提示工程在每个深度均达到饱和，战术原语从未超过 9%，与 (a) 形成镜像。行动表面是文本，因此这正是欧米茄发挥作用之处。

图 6：按深度划分的涌现层角色，按求解器基底拆分。每个条形图表示该深度 n 排放中（子标题中每深度 n ）两位评分者（GPT-5.2 与 Kimi-K2.6）均归入该类别的比例，覆盖深度 2 至 5 的 588 次排放。评分标准为多标签，因此同一深度内的条形图总和超过 100%，应跨深度比较而非相互比较。评分者意见分歧的排放被舍弃，使每个条形图均为下界。两个可忽略的类别被省略；评分标准及各类别 κ 见附录 E.3。

深度 2：跨任务传播的通用原语。在 CO-Bench 上，深度 2 的排放完全通用；两位评分者均将其全部 12 个 d2 排放标记为战术原语 (local_search, simulated_annealing, normalize_label, safe_log)。图 6a 中较低的 28% 来自另外两个代码档案的稀释，其中深度 2 更常进行路由。大多数 CO-Bench 获胜者最终调用这些原语，36 个获胜求解器中有 22 个调用了至少一个欧米茄排放的库函数。simulated_annealing 首次在单个候选中于深度 2 排放，随后传播至四个问题族的 36 个获胜者中的 15 个。每条链均改善 $d1 \rightarrow d2$ （平均提升 +0.113）；仅深度 2 就平均修复了每条链中 3.7 个先前失败的任务。

深度 3：专业化与层间干扰。深度 3 倾向于针对特定任务族的专业化处方，而 § 2.3 中预期的层间干扰在此首次出现，因为针对某一族调整的处方可能覆盖在其他地方有效的指导。在代码基底上，68% 的 d3 排放提供专用库，任务类别的 if/elif 路由随深度增强而非在此首次出现（图 6a）；总体 $d2 \rightarrow d3$ 平均提升仅为 -0.006，但逐任务差异很大，41% 的（链，任务）对在深度 3 严格退化，18% 大幅下降 ($d3 < 0.8 \cdot d2$)。On multi_demand_multidim_knapsack, 深度 2 的“贪心+local_search”提升 $0.0 \rightarrow 0.759$ 而深度 3 的“simulated_annealing 配合组转移移动”退化为 0.230；这类失败促使深度 4 的回滚行为。

更深层（4 至 6）：修正与细化。在深度 ≥ 4 处，纠正与细化作用持续存在。在代码基板上，回滚意图在两种深度下均占近半数的生成量，而专用库的使用持续上升；在提示基板上，回滚在深度 4 达到峰值。因此，更深层的作用是纠正和细化先前层，而非重新设计求解器。每个深度的平均得分在深度 3 达到峰值（0.726），并在深度 6 降至 0.602，但每个深度的平均值并非档案优化的目标。深层持续产生浅层无法达到的逐任务增益。

链达到。深度 ≥ 4 的候选方案在 CO-Bench 任务中获胜 31%，在 SR-matsci 中获胜 80%，在 SR-phys-osc 中获胜 70%。constrained_guillotine_cutting 首次在深度 5 开启（所有祖先得分为 0）；三个 matsci 任务首次在深度 6 开启。因此，档案同时保留了浅层回退方案（用于深度 3 的过度指定）和稀有的深层专家。

通过构造防止深度 3 的回归。干扰也可以被直接预防，而非在深度 4 以上修复。在 § 2.4 的整合模式下，每个候选方案改进一个焦点任务，并继承档案中冻结的最佳迹用于其余任务，因此逐任务最佳轨迹在构造上单调。在一个 8 任务 CO-Bench 频段（3 个种子）上，整合将逐任务最佳从 0.502 提升至 0.71 ± 0.02 ，并以 +0.10 (95% CI [+0.04, +0.16])，的优势击败计算匹配的 4 选 1 对照，在 8 个任务中的 8 个上实现零回归。因此，我们可以将 § 2.3 中的组合覆盖声明视为经过测量而非仅仅断言；档案实现了单一链丢弃的逐任务增益，并且可以单调地实现（完整研究见附录 G）。

框架是通用的；角色进展自适应。单一的 Ω 模板在所有八个基准上运行，但涌现的角色进展会根据每个基准的失败模式进行调整。在失败模式多样化的基准上（CO-Bench、SR-matsci、SR-phys-osc）最为显著；仅提示的基准（S2D、LawBench）在每个深度都在提示工程空间内进展，因为动作表面仅为文本。

框架的收益所在。元 (Meta) n 的增益程度反映了基准测试可能失败的不同方式的数量。CO-Bench 的 36 个 NP 难形状和 SR-matsci 的 25 条不同定律各自呈现出许多正交的失败模式，因此跨任务代码库和逐深度专业化共同作用，产生了我们所观察到的最大差距；TB2 的 13 个沙盒类别和 AlphaEvolve Math 的混合数值组合任务也表现出相同的行为。ARC-AGI-2 是一个极端案例，基线几乎没有变化，而递归产生了唯一高于底线的条目。S2D 则处于另一端，因为其较小的动作空间（一个提示，22 个标签）使得元 n 相对于 OE 的领先优势在两个骨干网络上都在种子噪声范围内。开发集到测试集的梯度显示出相同的排序。CO-Bench（开发集 0.886，测试集 0.870）将存档结构迁移到留出任务上；S2D（开发集 0.887，测试集 0.725）在提示重写契约坍塌到单一留出分布时趋于饱和；ARC-AGI-2 在绝对数值下降的情况下仍保留了分类上的高于底线信号。

在它不起作用的地方。第二个条件是种子与上限之间的空间，有两个基准测试处于其极限。AlgoTune 是过度约束的情况，其中预优化的核函数契约几乎没有给 Ω 留下可添加的内容，其额外的上下文反而使代码更紧致而非改进它。大部分减速来自两个 FFT 核函数，而不是分散在整个基准测试中（附录 I），因此可以逐任务检查诊断，而不是依赖单一聚合数字。在 SWE-Bench（ $B=K=2$ ，三次迭代）上，种子已经足够强，存档中的最佳候选是第 0 代候选， Ω 从未被激活。因此， Ω 的价值与两个量都成比例：种子之上的余量和存档可以索引的失败模式多样性。

3.4 消融实验

条件化是主导通道。移除层间上下文（在层之间传递的单个字符串）解释了递归相对于深度 1 基线增益的 $\sim 72\%$ 。更丰富的代码库通道（可调用的 Python 函数）解释了 $\sim 15\%$ ；剩余的 $\sim 13\%$ 来自递归机制本身（束搜索、重试、启发）（表 3）。

表 3：组件消融。每一行移除一个机制并保留其余部分，比较在块内进行同类比较。

Condition	Removal	Score	Δ
Meta ⁿ (full; Gemma, CO)		0.845	
–code library	$\mathcal{L}^{(d)}$ injection	0.825	–0.020
–outer-context	inter-layer ctx _d	0.751	–0.094
–recursion (depth-1)	meta-layer stacking	0.714	–0.131
Meta ⁿ (full; GPT-5.2, CO)		0.886	
–recursion (depth-1)	meta-layer stacking	0.806	–0.080
Meta ⁿ (full; GPT-5.2, CO)		0.917	
–recursion (depth-1)	meta-layer stacking	0.759	–0.158

递归与扁平替代方案的对比。第 3.2 节的基线每个基线移除架构的不同部分。GA 移除了递归，使其修正配置下，且在 $10\times$ 预算下，它在 GPT-5.2 CO-Bench 上仍停滞于 0.628，远低于 Metaⁿ 的 0.870，这表明该差距是 GA 架构的极限，而非其计算量的极限。OE 保留了搜索循环，但按任务运行，因此其在两个骨干网络上的 CO-Bench 测试差距（表 1 和表 2）即为跨任务迁移的价值，该价值在 CO-Bench 上以匹配计算量衡量，在提示基准上以受限计算量衡量（第 3.2 节）。在 Metaⁿ 内部，深度为 1 的消融仅移除了递归。其提升在不同骨干网络和基准上均可复现（ $+0.131$ Gemma CO-Bench、 $+0.080$ GPT-5.2 CO-Bench、 $+0.158$ GPT-5.2 AE Math），并随着内循环上限的降低而增大，这与第 2.3 节的预测一致。

深度增益的来源。消融分解表明，上层策略字符串在每一步重塑下层行为，且这种条件作用带来的增益大于该层同时传递的可调用代码。这可以视为第 2.3 节中 $\prod_d^{k_d}$ 论证的经验形式。深度通过条件作用倍增覆盖率，存档将覆盖率转化为逐任务收益（表 1 和表 2 中存档相对链的边际），而整合研究（第 3.3 节，附录 G）表明这些收益可以单调累积。

4 结论

本文提出了 Metaⁿ，一种自改进智能体，其构建方式是固定一个元操作 Ω ，并对其输入而非操作本身进行递归。每次应用 Ω 时，都会读取下方堆栈的迹以及生成这些迹的代码，因此其起始信息集严格大于前一次应用，堆栈不断加深直至改进停止。这化解了先前工作所面临的困境——递归改进器只能以牺牲稳定性为代价换取深度，而本文方法无需编辑改进器即可实现。

证据在测试的两个轴向上均支持该架构。在八个基准家族和两种骨干网络上，元ⁿ 在至少一个估计器上领先于每个基准家族，且在基准以多种不同方式失败时领先幅度最大。在旨在抵抗技能记忆的 ARC-AGI-2 上，对象级迭代保持在接近下限的水平，而元级堆栈达到 0.331。组件消融比预期更精确地定位了增益来源。递归带来的大部分收益来自层间最朴素的通道——传递的上下文字符串，约占总提升的 72%，而可调用代码传递约占 15%。随后，角色在没有明确提示规定的情况下在深度上自然涌现。

局限性与未来工作。我们在基础求解器和每次 Ω 调用中运行相同的模型。这是分离深度增益与能力增益的正确对照，但未测试实际部署场景，即更强的模型位于 Ω 之上而较弱的基础模型在下；当仅升级改进器时深度是否仍能持续获益，是自然的下一步实验。此外，由于条件化而非代码传递贡献了大部分提升，我们可以进一步丰富层间传递的上下文，例如用结构化或类型化表示替代自由格式字符串。另一个方向是研究堆栈可以有效地达到多深。我们观察到运行在深度 3 到深度 6 之间停止，因为 Ω 停止发现改进，而不是因为基础模型在更高元级别上耗尽推理能力或上下文被累积的层代码填满。

致谢

我们感谢明尼苏达自然语言处理小组成员在组会中提出的深刻评论。ZMK 得到 3M 科学技术奖学金和明尼苏达大学博士论文奖学金的大力支持。

参考文献

- Rakefet Ackerman and Valerie A. Thompson. Meta-reasoning: Monitoring and control of thinking and reasoning. *Trends in Cognitive Sciences*, 21(8):607–617, 2017. doi: 10.1016/j.tics.2017.05.004.
- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alex Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=RQm2KQTM5r>.
- Xinyun Chen et al. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.
- Francois Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. Arc-agi-2: A new challenge for frontier ai reasoning systems, 2026. URL <https://arxiv.org/abs/2505.11831>.
- Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktaschel. PromptBreeder: Self-referential self-improvement via prompt evolution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- Douglas R. Hofstadter. *Gödel, Escher, Bach: An Eternal Golden Braid*. Basic Books, New York, NY, 1979. ISBN 0-465-02685-0.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations (ICLR)*, 2025.
- Seungyeon Jwa, Daechul Ahn, Reokyoung Kim, Dongyeop Kang, and Jonghyun Choi. Becoming experienced judges: Selective test-time learning for evaluators. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 697–721, 2026.
- John Kounios and Mark Beeman. The cognitive neuroscience of insight. *Annual Review of Psychology*, 65:71–93, 2014. doi: 10.1146/annurev-psych-010213-115154.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint*, 2026.
- Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. Oral, Top 1.5%.

- Shu Liu, Shubham Agarwal, Monishwaran Maheswaran, Mert Cemri, Zhifei Li, Qiuyang Mang, Ashwin Naren, Ethan Boneh, Audrey Cheng, Melissa Z. Pan, Alexander Du, Kurt Keutzer, Alvin Cheung, Alexandros G. Dimakis, Koushik Sen, Matei Zaharia, and Ion Stoica. EvoX: Meta-evolution for automated discovery, 2026. URL <https://arxiv.org/abs/2602.23413>.
- Aman Madaan, Niket Tandon, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- Yuxiao Qu, Tianjun Zhang, Naman Garg, and Aviral Kumar. Recursive introspection: Teaching language model agents how to self-improve. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Bernardino Romera-Paredes, Mohammadamin Barekatin, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- Stuart Russell and Eric Wefald. Principles of metareasoning. *Artificial Intelligence*, 49(1–3):361–395, 1991. doi: 10.1016/0004-3702(91)90015-C.
- Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint arXiv:cs/0309048*, 2003.
- Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. AgentSquare: Automatic LLM agent search in modular design space. In *International Conference on Learning Representations (ICLR)*, 2025.
- Asankhaya Sharma. OpenEvolve: an open-source evolutionary coding agent. GitHub, 2025. URL <https://github.com/algorithmicsuperintelligence/openevolve>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Weiwei Sun, Shengyu Feng, Shanda Li, and Yiming Yang. CO-Bench: Benchmarking language model agents in algorithm search for combinatorial optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. ReEvo: Large language models as hyper-heuristics with reflective evolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Xunjian Yin, Xinyi Wang, Liangming Pan, Xiaojun Wan, and William Yang Wang. Godel Agent: A self-referential agent framework for recursive self-improvement. *arXiv preprint arXiv:2410.04444*, 2024.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. In *Conference on Language Modeling (COLM)*, 2024.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Godel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025a.
- Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. HyperAgents. *arXiv preprint*, 2026.
- Jiayi Zhang, Jinyu Xiang, et al. AFlow: Automating agentic workflow generation. In *International Conference on Learning Representations (ICLR)*, 2025b. Oral, Top 1.8%.

A 符号说明

表 4 汇总了 § 2 中使用的符号。

表 4: 第 2 节符号表。符号 简明含义 固定元操作, 在每个构建步骤调用

Symbol	Plain English
Ω	fixed meta-operation, called at each build-step
M_d, S_d	depth- d wrapper and resulting solver
C_d	Ω 's output: pre-process $f_{\text{pre}}^{(d)}$ + library $\mathcal{L}^{(d)}$
$\tau_i^{(d)}$	execution trace at depth d on task i
ctx_d	outer-context passed from layer d to layer $d-1$

B 相关工作

单层 自改进与优化。单反馈环系统 (Self-Refine [马丹等人, 2023]、Reflexion [希恩等人, 2023]、Self-Debugging [陈等人, 2024]、RISE [曲等人, 2024]) 执行生成、评估、反思、重试。进化方法维护种群。FunSearch [罗梅拉-帕雷德斯等人, 2024] 将冻结的大语言模型与评估器配对, 在孤岛式循环中运行; AlphaEvolve [诺维科夫等人, 2025] 与 OpenEvolve [夏尔马, 2025] 将其推广到整个代码库; 启发式进化 [刘等人, 2024] 共同进化自然语言“思想”与代码; ReEvo [叶等人, 2024] 引入“语言梯度”。架构搜索系统 (ADAS [胡等人, 2025]、AgentSquare [尚等人, 2025]、AFlow [张等人, 2025b]) 从存档中编程新智能体, 或在代码工作流上运行蒙特卡洛树搜索。所有这些都在单一抽象层次上运行, 其中生成、变异或搜索的机制本身是固定的; 批评者从未被批评, 进化循环也从不进化其自身的选择规则。Meta n 通过堆叠元层来解除这一限制, 每一层生成修改下一层的代码, 并接受上一层的评估; 堆叠深度由收敛性决定。

自指智能体。施米德胡贝的哥德尔机 (Gödel Machine) [施米德胡贝, 2003] 为可证明的自我改进智能体提供了理论基础 (形式化证明要求限制了其实用性)。STOP [泽利克曼等, 2024] 展示了递归式脚手架改进, 但仅实现了一个元步骤。哥德尔智能体 (Gödel Agent) [尹等, 2024] 重写自身源代码; 达尔文哥德尔机 (Darwin Gödel Machine) [张等, 2025a] 将进化种群维护与自指修改相结合, 在 SWE-bench 上达到 50%; 超智能体 (HyperAgents) [张等, 2026] 将任务智能体与元智能体合并为一个可编辑程序。EvoX [刘等, 2026] 将进化程序搜索扩展到元进化机制, 其中搜索算子自身也在进化。所有这些经验上最多实现 ~ 2.5 个元层级, 因为至少有一个驱动层为保持稳定性而被固定。哥德尔智能体的动作应用程序编程接口 (API) 和目标提示受到硬性保护; 达尔文哥德尔机明确“将存档维护和父代选择……固定且不可由达尔文哥德尔机修改” (第 3 节); 超智能体允许内部例程编辑, 但其外部选择与评估循环“不可更改” (第 7 节)。该比例反映了求解器的修改 (层级 1) 以及修改逻辑本身的修改 (层级 2, 因为该逻辑位于被编辑的源代码内部) 均已完全实现, 而每个更高层级仅对机制中可编辑部分进行更改, 从不触及冻结的驱动层, 因此最多获得半级信用。

元脚手架。元框架 (Meta-Harness) [李等, 2026] 通过将先前候选者的源代码、得分和迹的完整历史暴露给编码智能体 ($\sim 10\text{M}$ 个诊断上下文标记) 来优化大语言模型周围的代码基础设施。提示育种器 (PromptBreeder) [费尔南多等, 2024] 同时进化任务提示和变异提示 (提示空间中的两级自指), 而 GEPA [阿格拉沃尔等, 2026] 表明仅靠反思性提示进化就能超越强化学习微调, 同时 LWE [朱瓦等, 2026] 将反思循环应用于评估器本身, 在评估过程中无标签地改进大语言模型评判器。这些方法将驱动层置于可编辑表面之外 (元框架), 最多实现两个元步骤 (提示育种器), 或在提示空间而非代码空间中操作。

元 (Meta) n 所增加的内容。结构重构带来四个优势: (i) 分层次结构而非角色折叠; (ii) $\prod_{d=2}^n k_d$ 组合覆盖而非 $\sum_{d=2}^n k_d$, 因为每个 Ω 层

重塑了所有下层层执行方式；（三）通过收敛而非手工设置 T 或开放式存档增长实现涌现深度；（四）在八个异构基准上使用单一 Ω 提示模板（每个基准调整编排超参数），而非针对每个基准进行提示或算子搜索。经验上，实现的深度从 2 到 6 不等，并伴随涌现的角色专门化（深度 2 的通用原语跨任务传播，深度 3 的任务感知路由，更深层的专门化与回退），据我们所知，这首次证明了元深度 > 2 产生结构上不同的层次而非冗余层次。

C 实验设置细节

表 5：两个骨干网络上每个基准的进化超参数。 Ω 提示模板、输出模式、 $\epsilon = 0.02$ 和 $\alpha = 0.3$ 在所有行中固定不变。最大深度为 10，TB2 除外（4）。“关闭”表示禁用早停，运行至最大迭代次数。GPT-5.2 列在 CO-Bench（8 到 6）和 S2D（20 到 10）上收紧了最大迭代次数以限制每个种子的成本，AE Math 和 SR 保持为 6；TB2 按类别逐一扫描。ARC-AGI-2 仅在 GPT-5.2 下运行（种子 42/43/44； $\sim$ 每个 Meta n 种子 632 美元）。

基准	B	K	Pat.	最大迭代次数		Test split?
				Gemma	GPT-5.2	
CO-Bench	2	2	4	8	6	yes
Symptom2Disease	1	3	off	20	10	yes
LawBench	1	3	off	20	n/r	yes
TerminalBench 2.0	1	2	3 to 4	6 to 10	13 cat-by-cat	no
AlphaEvolve Math	2	2	10	6	6	no
Symbolic Regression	2	2	10	6	6	no
AlgoTune	2	2	10	6	n/r	no
ARC-AGI-2	2	2	4	n/r	8	yes

基准。（一）组合优化基准（**CO-Bench**）[Sun 等人，2026]：包含 36 个非确定性多项式难（NP-hard）组合优化问题，采用 Python 求解器，评分范围为 $[0, 1]$ 。（二）症状到疾病（**Symptom2Disease**, S2D）：22 类医学诊断任务，以准确率为度量指标；Python 求解器内置环境大语言模型（llm0）辅助函数。（三）法律基准（LawBench）罪名预测：中文法律多标签分类任务，以 F1 分数为度量指标；同样采用 Python 与大语言模型（llm0）接口格式。（四）终端基准 2.0（TerminalBench 2.0, TB2）：包含 89 个终端智能体任务，覆盖 13 个类别，运行于 Docker 沙箱环境中，采用 bash 求解器；同时提供 bash 函数与 Python 辅助脚本。（五至七）开放进化基准（OpenEvolve）：包括 AlphaEvolve 数学（数学发现任务；问题数量随设置变化，详见下文协调说明）、符号回归（Symbolic Regression, SR；从数据中发现方程，涵盖 4 个领域）、算法调优（AlgoTune；算法加速）；所有任务通过共享适配器集成，支持问题目录自动发现与子进程隔离评估。（八）抽象推理语料库-通用人工智能-2（**ARC-AGI-2**）：包含 120 个抽象推理网格谜题，求解器需从少量输入/输出示例中推断变换规则，以两次尝试内通过率（pass@2）评分；仅在 GPT-5.2 模型下运行。

AlphaEvolve Math 问题数量协调。 AE Math 任务池随着新问题被移植到 OpenEvolve 适配器中而迭代扩展。Gemma 单次运行在运行时可用的 15 个问题进行了评估；Gemma 智能体运行以及三种子 GPT-5.2 Meta n 和 Gödel Agent 运行在 16 个问题进行了评估；OpenEvolve GPT-5.2 运行在 10 个问题的子集上进行了评估，因为其基于差异的变异需要每个问题的起始求解器，而三个较新的问题缺少起始求解器。AE Math 的汇总数字使用每个系统可用的最大问题集。

求解器和编排器超参数。 基础求解器温度 0.3； Ω 温度 0.7（线性）或循环 $\{0.5, 0.7, 0.9\}$ （进化）。线性： $\epsilon = 0.02$ （注入被视为进展所需的最小平均得分增益）， $D = 10$ （最大深度上限），10 秒评估超时。进化： Ω 提示模板、输出模式、 $\epsilon = 0.02$ 和 $\alpha = 0.3$ （在父采样中对已扩展链进行降权的探索奖励）在所有基准上固定；每个基准的束宽 B 、分支 K 、耐心和最大迭代次数见表 5。自调试允许 `max_retries =` 为 2，并在 CO-Bench 和分类上使用与基准相关的 `retry_threshold` (0.1, 0.3 在

AlphaEvolve Math 和 AlgoTune 上，0.5 在符号回归和 TB2 上）。单次运行允许最多两次执行器级重试；智能体运行允许最多 8 个观察后行动回合。两种变体共享相同的进化超参数。

墙钟成本。完整的进化运行通过耐心或存档饱和终止；两者均无先验界限。在 CO-Bench（36 个任务）上，单次运行在驱动 OpenRouter API 的单台机器上耗时 ~ 3 小时，主要由第一层评估而非 Ω 调用主导，智能体变体耗时 ~ 10 小时（每个任务最多 8 次 LLM 调用）。TB2 的墙钟时间主要由 Docker 启动主导；89 个任务的智能体运行耗时 ~ 28 小时。花费随表 6 中的每行令牌预算扩展。

表 6：七个 Gemma 基准系列的每行深度、存档大小和令牌花费，包括图 4 的四个面板（s42 生产运行）。

Benchmark	Mode	Depth	Cands	Tokens
CO-Bench (36 tasks)	Single-shot	3	33	5.3M
	Agentic	3	29	36.1M
Symptom2Disease	Single-shot	5	61	422K
	Agentic	3	61	1.7M
LawBench (Chinese)	Single-shot	4	61	761K
	Agentic	4	61	2.3M
TerminalBench 2.0 (89)	Single-shot	4	92	3.7M
	Agentic	3	115	35.6M
AlphaEvolve Math (15/16)	Single-shot	3	25	2.9M
	Agentic	4	24	25.7M
Symbolic Regression (4)	Single-shot	3 to 6	97	12.1M
	Agentic	3 to 4	100	21.7M
AlgoTune (8)	Single-shot	3	25	1.1M
	Agentic	4	24	6.9M

基线 配置（Gemma）。哥德尔智能体（Gödel Agent, GA）：20 次迭代， $\text{max_val} = 50$ ， $n_{\text{few_shot}} = 10$ ， $\text{solve_timeout} = 300$ 秒；与 Meta n 使用相同的基础模型。我们在 CO-Bench、Symptom2Disease、LawBench、AlphaEvolve Math 和 AlgoTune 上运行 GA。TB2 被排除，因为在其上运行 GA 需要对其测试框架进行大量修改；SR 基线仅在 GPT-5.2 下运行。所有报告的 CO-Bench GA 数值均使用修正后的逐任务配置；已发布的单求解器接口和预算对等研究记录在附录 J 中。**OpenEvolve** (OE)：三个岛屿，MAP-Elites 10×10 特征网格，**OpenEvolve** 的默认温度调度；使用相同的基础模型。CO-Bench 对每个任务运行 36 次独立进化，每次 10 次迭代，并采用基于差异的代码编辑；Symptom2Disease 和 LawBench 对每个任务运行一次完整重写提示进化，共 20 次迭代。第 3.2 节的计算对等运行将 Meta n 的令牌消耗硬性限制为每个基准测试测得的 OE 预算。

基线配置（GPT-5.2）。我们在 CO-Bench、Symptom2Disease、AlphaEvolve Math、SR 和 ARC-AGI-2（种子 42/43/44）上使用 Gemma 超参数重新运行了 GA 和 OE，并与表 2 的 Meta n 单元格进行对比。唯一的偏差是表 5 中 GPT-5.2 列的成本受限迭代上限。LawBench 和 AlgoTune 未在 GPT-5.2 下运行。

深度-1 Meta n 消融（GPT-5.2）。我们将存档折叠为单层，无层间上下文、无代码库注入、无递归； $n_{\text{runs}} = 4$ 每个种子在温度 1.0 下，种子 42/43/44， ~ 870 K 令牌每个种子。未运行留出测试，因此表 3 中的深度-1 锚点是存档最佳验证分数（四次运行中每个任务的最佳值，跨任务平均），与该表中全栈锚点使用的估计器一致。

D 设计决策与理由

所有层使用相同模型。我们在基础求解器和每次 Ω 调用中使用相同的大语言模型（主要运行使用 Gemma 4 31B-IT；第 3 节的跨模型研究使用 GPT-5.2）。这隔离了递归的贡献与模型异质性的贡献；深度- d 的改进不能归因于更强的改进器。异质混合（例如在 Ω 处使用更强的模型）是明显的下一步（第 4 节），但会使消融混杂。

包装器，而非补丁。 Ω 生成的代码通过非侵入式元层（MetaLayer）包装器注入；除名称覆盖外，它无法改变内部求解器的状态、任务对象或先前层的库。这排除了一类自修改智能体破坏自身脚手架的错误（该失败模式在 DGM 第 3 节中报告，也是 Gödel 智能体采用硬保护动作应用程序编程接口的动因）。

安全性与沙箱化。Meta n 分两个阶段验证所有 Ω 生成的代码。静态分析：（1）通过 `ast.parse` 进行语法检查，（2）AST 遍历以检测危险导入（`os`、`subprocess`、`socket`、`sys` 等）和被阻止的属性，（3）代码长度限制（10,000 字符）。冒烟测试：代码库函数在隔离命名空间中执行，以验证它们定义了具有预期名称的可调用对象；在定义时崩溃的函数（例如由于缺少导入）将被跳过并发出警告，而不是前置到求解器的脚本中。

3:1 失败偏置迹比率。试点日志显示，深度 2 和 3 的 Ω 将大部分有用推理用于诊断失败模式；成功迹仅作为正例样本。我们在 10 任务 CO-Bench 试点上调整了 `failure_ratio`；0.5 会稀释失败信号，1.0 会剔除正例样本并产生过拟合约束。0.75 是保留正例样本作用的最小比率。

收敛阈值 $\epsilon = 0.02$ 。低于此阈值时，增益处于我们基础模型在 CO-Bench 上单种子运行间方差范围内，因此进一步加深是不合理的。进化模式使用相同的 ϵ 来定义非改进迭代；耐心在连续 F 次此类迭代后触发。

波束设置。对于父代差异显著的基准（CO-Bench、AlphaEvolve Math、AlgoTune、SR），我们设置 $B = K = 2$ （允许交叉授粉的最小波束）。分类（S2D、LawBench）使用 $B = 1, K = 3$ ，TB2 使用 $B = 1, K = 2$ ，因为仅提示和沙箱绑定的基准从更宽的父代波束中获益较少。更大的波束使令牌成本线性扩展，而不改变定性发现：档案为任何单一链开辟了每任务余量。

实用深度。表现最佳的链跨越深度 3 到 6（在大多数基准上在 3 到 4 处达到平台期；在 `matsci` 符号回归上深度为 6）。最优深度跟踪失败模式多样性。两个上限尚未测试：模型在更高元层级上的推理能力，以及累积层代码导致的上下文窗口饱和。

进化档案的价值。档案将深度与质量解耦（深度 2 的候选可以优于深度 3），并实现跨候选策略转移；这些共同开辟了超越任何单一链的每任务余量。

E 扩展方法论

E.1 进化编排器

Algorithm 1 Evolutionary Meta-Recursion

Require: Tasks $\mathcal{T}$ with $N = |\mathcal{T}|$; base solver S_1 ; max depth D ; beam B , children K ; patience P ; exploration α . Chain mean $\bar{S}(c) = \frac{1}{N} \sum_t \text{score}(t, c)$; archive $\mathcal{A}$ of (chain, mean) pairs; archive-best $\bar{S}^* = \frac{1}{N} \sum_t \max_{c \in \mathcal{A}} \text{score}(t, c)$; non-improving counter p .

- 1: $\mathcal{A} \leftarrow \{(S_1, \bar{S}_1)\}$; $\bar{S}^* \leftarrow \bar{S}_1$; $p \leftarrow 0$
- 2: **while** $p < P$ **do**
- 3: Sample B parents from $\{c \in \mathcal{A} : \text{depth}(c) < D\}$ with $w(c) \propto \bar{S}(c) + \alpha/(1+\text{children}(c))$
- 4: **for** each parent $c_p, k = 1, \dots, K$ **do**
- 5: $C \leftarrow \Omega(c_p)$ $\triangleright T$ cycled through $\{0.5, 0.7, 0.9\}$; rival traces appended for tasks where c_p underperforms
- 6: **if** $C = \emptyset$ **then continue**
- 7: **end if**
- 8: $\mathcal{A} \leftarrow \mathcal{A} \cup \{(\text{METALAYER}(C, c_p), \bar{S}_{\text{new}})\}$
- 9: **end for**
- 10: $\bar{S}_{\text{cur}} \leftarrow \frac{1}{N} \sum_t \max_{c \in \mathcal{A}} \text{score}(t, c)$
- 11: $p \leftarrow 0$ **if** $\bar{S}_{\text{cur}} > \bar{S}^*$ **else** $p+1$; $\bar{S}^* \leftarrow \max(\bar{S}^*, \bar{S}_{\text{cur}})$
- 12: **end while**
- 13: **return** $\arg \max_{c \in \mathcal{A}} \bar{S}(c)$; archive-best $\bar{S}^*$

E.2 迹采样与深度感知载荷

Ω 的大语言模型调用模板（系统提示、输出模式、解析）在每个深度均相同；仅其摄入的迹载荷根据可用信号进行调整。每次调用最多看到 20 条迹，按 3:1 的失败与成功比例抽取（上下文窗口的 65% 保留给迹，35% 保留给先前的代码栈；溢出时最旧的层首先被丢弃）。

深度 ≤ 2 ：原始逐任务迹。在深度 2 时，载荷为逐任务格式化迹加上一个基线比较标志，标记改进或退化的任务。每条迹包含生成的脚本、标准输出/标准错误（截断至 500 字符）、退出代码、错误摘要以及评估器反馈字符串。

深度 ≥ 3 ：结构化摘要。在深度 ≥ 3 时，原始迹被替换为结构化摘要：（1）任务类别性能分解，（2）失败模式分布，（3）针对深度 $d-2$ 衡量的上一层有效性分析，以隔离其贡献，以及（4）三条代表性迹。在进化模式下，退化的任务还额外标注了存档中每任务的最佳结果，为 Ω 提供了明确的改进上限。

因此，深度感知格式化器是位于 Ω 之上的内容适配器，而非独立的驱动器；随着深度增长， Ω 的提示模板和输出解析器保持不变。

E.3 模式分类标准与评分者间一致性

在 CO-Bench、S2D、LawBench、AlphaEvolve Math 和 SR 生产存档中，共 596 条 Ω 输出（深度 2 有 119 条，深度 3 有 203 条，深度 4 有 180 条，深度 5 有 86 条，深度 6 有 8 条）被分类到一个或多个类别中，共七类：（i）环境约束（例如，库排除）；（ii）战术原语（可调用的库函数，如 `local_search()`）；（iii）任务类别路由（基于任务类型或主题的 `if/elif`）；（iv）专用库（任务类型化的辅助函数，如 `geometry_utils()`）；

（v）提示工程编辑（角色、语言、思维链脚手架）；（vi）回滚或覆盖（理由文本中引用“退化”、“回滚”、“过度约束”、“放弃”）；（vii）组合架构（多功能流水线）。

两个独立提示的大语言模型评估器，GPT-5.2 和 Kimi-K2.6，根据该评分标准标注了全部 596 条排放。图 6 报告了在每个深度上两个评估器都标记为匹配某类别的排放比例；一条排放可能匹配多个类别，因此同一深度内的比例之和不为 100%。图中省略了质量可忽略不计的两个类别。环境约束是一种浅层代码现象，随深度衰减（在深度 2 至 5 的代码基底排放中分别占 13%、11%、3%、9%，即 9、14、2 和 2 条排放），这与 §1 工作示例中深度 2 的依赖变通方案一致。组合式架构在所有深度均低于 14%，其在深层代码层上的表面上升

仅基于 23 条排放中的 3 条，因此我们不将其解读为趋势。表 7 给出了每个类别的 Cohen's κ 。当线索接近句法层面时（任务路由、提示工程），一致性接近完美；而在两个最抽象的角色上一致性最弱，其中 GPT-5.2 的标注比 Kimi 更宽松。§ 3.3 的两个主要模式，即纠正角色的深度涌现和代码与文本的分化，在任一评估器单独标注时均可复现。

表 7：每个评分标准类别的评估者间一致性 (Cohen's κ ，GPT-5.2 对比 Kimi-K2.6，全部 596 条排放)。

Category	κ
Task routing	0.96
Prompt engineering	0.85
Rollback / override	0.60
Specialized libraries	0.57
Environment constraints	0.45
Tactical primitives	0.37
Compositional architectures	0.30
Mean over the 7 categories	0.59

F 深度 2 注入代码（完整）

以下是 Ω 在试点实验中于深度 2 生成的完整预处理代码：

```
# 解决“没有名为 'scipy' 的模块”和超时模式。additional_context = """ ###
重要环境限制：1. 外部库：除 'numpy' 外，请勿使用 'scipy' 或任何第三方
库。如果需要线性规划、分配算法（如匈牙利算法）或优化求解器，必须
使用纯 Python 或 'numpy' 从头实现。2. 性能：避免在大坐标范围上使用嵌套
循环。使用坐标压缩、基于事件的处理或带有稀疏数据结构的贪心启发式
方法以避免超时。3. 验证：确保所有输出索引严格符合任务描述中指定的索
引要求。 """
```

这段由 Ω 通过失败迹分析自动生成的代码，通过防止 10 个任务中 4 个任务最常见的失败模式，将平均得分从 0.439 提高到 0.513（相对提升 16.9%）。

G 巩固防护：研究与控制

更深的层可以回归单个任务（§ 2.3）；一个在某个任务族上误触发的深度-3 路由器会覆盖原本有效的深度-2 指导。§ 2.4 的巩固模式通过构造来防止这种情况。每个候选者针对单一焦点任务，并为所有其他任务继承档案中冻结的最佳轨迹，因此每个任务的最佳轨迹永远不会下降。

研究。在一个 8 任务的 CO-Bench 频段（3 个种子）上，巩固模式将每个任务最佳均值从 0.502 提升到 0.71 ± 0.02 ，在 8 个任务中的 8 个上实现了零任务回归。一个计算匹配的对照，即在标准编排器下进行 4 个种子重采样的最佳结果，达到 0.61 ± 0.03 ；相对于对照的 +0.10 提升带有 95% 置信区间 $[+0.04, +0.16]$ 。一个较早的单种子试点（6 任务子集，8 代，本地服务的量化 Gemma 骨干）显示出相同的零回归行为（ $0.502 \rightarrow 0.685$ 对比对照 0.674）。

我们声明的内容。在这个频段上，巩固模式在均值提升上也超过了计算匹配的对照 (+0.10，置信区间不包括零，但我们将声明建立在轨迹的形状上；未受保护的编排器在深度-3 路由误触发时以显著的成对速率产生任务回归，而巩固模式通过构造是单调的，并且永远不会回归一个已解决的任务。因此，在回归比均值爬升更昂贵的部署中，这种保护最为重要。

H 符号回归各领域分解

表 8 将表 1 和表 2 中的 SR 条目扩展为四个领域的演变。在 Gemma 生产运行中，单次射击在 chem_react 和 phys_osc 上赢得档案最佳，其更强的种子留下了有限的空间；智能体在 bio_pop_growth、材料科学和家族均值上获胜。在 GPT-5.2（表 9）下，Meta n 在每个领域都领先于两个基线。Gödel Agent 的大标准差（例如，chem_react 2.92 ± 3.84 ）来自种子级别的崩溃，其中其单一自我修改使求解器回归。

表 8：符号回归各领域结果（Gemma，s42 生产运行），支持表 6 中的符号回归行。领域：bio_pop_growth（生物种群动态），chem_react（化学反应动力学），matsci（材料科学性质），phys_osc（物理振荡器系统）。底部块给出家族均值（四个运行中的令牌求和；其他列取平均）。粗体=表示每个领域更高的档案最佳值。

Domain	Mode	Seed	Linear-best	Archive-best	Depth	Cands	Tokens
bio_pop_growth (24 inst.)	Single-shot	0.477	2.751	3.782	3	25	2.6M
	Agentic	2.355	4.406	4.977	3	25	5.2M
chem_react (36 inst.)	Single-shot	5.458	6.580	7.599	3	25	2.6M
	Agentic	5.587	6.440	7.244	3	25	2.7M
matsci (25 inst.)	Single-shot	0.000	1.273	2.130	6	23	3.9M
	Agentic	1.477	3.296	3.863	3	25	6.7M
phys_osc (44 inst.)	Single-shot	2.686	4.191	4.681	4	24	3.0M
	Agentic	3.114	3.573	4.514	4	25	7.1M
<i>Mean across 4 domains</i>	Single-shot	2.155	3.699	4.548	n/a	n/a	12.1M
	Agentic	3.133	4.429	5.149	n/a	n/a	21.7M

表 9：GPT-5.2 下的符号回归各领域比较（智能体档案最佳，均值 $\pm$ 种子间标准差 42/43/44）。)

Domain	Meta ⁿ	OpenEvolve	Gödel Agent
bio_pop_growth	4.35 $\pm$ 0.30	2.83 $\pm$ 0.08	2.68 $\pm$ 0.83
chem_react	7.65 $\pm$ 0.07	6.96 $\pm$ 0.08	2.92 $\pm$ 3.84
matsci	3.26 $\pm$ 0.67	1.37 $\pm$ 0.34	1.27 $\pm$ 0.67
phys_osc	4.85 $\pm$ 0.11	3.63 $\pm$ 0.15	2.93 $\pm$ 2.03
Mean	5.03 $\pm$ 0.20	3.70 $\pm$ 0.13	2.45 $\pm$ 1.63

I AlgoTune 逐任务分解

表 10 将 AlgoTune 聚合结果分解为两个 Meta n 变体（单种子，s42）的逐任务加速比。反转在两个快速傅里叶变换核函数上最为显著，仅这两个核函数就贡献了七个共享任务中总共损失的 17.55 加速比中的 11.08。另外两个任务在智能体变体下损失了三分之一到一半的加速比，psd_cone_projection（5.54 降至 3.74）和 affine_transform_2d（2.00 降至 1.05），其余三个任务接近持平。在单次运行中，深度 2 的规范性提示使 fft_convolution 回退至 $1.11\times$ ，深度 3 的推理诊断出原因——缺少内存连续性约束——并将其修复至 $9.72\times$ 。智能体运行从未恢复同一核函数。这是 § 3.3 中过度约束诊断的任务级证据。

表 10: AlgoTune 逐任务加速比 ($\times$, s42)。两个变体均解决的七个任务。第八个任务 `eigenvectors_complex` 仅在智能体运行中有逐任务日志 ($\times 1.11$)，因此单次聚合 ($\times 18.47$) 平均七个任务，智能体聚合 ($\times 14.11$) 平均八个任务；在匹配的七个任务上，智能体均值为 $\times 15.96$ 。

Task	Single-shot	Agentic
<code>convolve2d_full_fill</code>	105.37	101.79
<code>fft_convolution</code>	9.72	1.69
<code>fft_cmplx_scipy_fftpack</code>	4.17	1.12
<code>psd_cone_projection</code>	5.54	3.74
<code>affine_transform_2d</code>	2.00	1.05
<code>lu_factorization</code>	1.47	1.34
<code>polynomial_real</code>	1.03	1.02

J Gödel 智能体基线细节

两种配置。GA 已发布的框架从通用数学问答模板中播种单个求解器函数，并要求其覆盖全部 36 个异构 CO-Bench 任务。该种子不生成任何逐实例的 `solve()`，因此运行在两个骨干模型上均坍缩为 ~ 0.000 ；坍缩是接口的特性而非模型的特性，并且在不同种子间完全重现（在 GPT-5.2 上使用已发布接口，每个种子的存档最佳均值字节级相同，标准差 $= 0$ ）。将单个标志切换为逐任务模式，使用相同的智能体、模型和评估器，为每个任务提供独立的求解器谱系，并在 Gemma 上恢复 0.451 ± 0.023 ，在 GPT-5.2 上恢复 0.527 ± 0.033 。主表中的所有 GA 数字均使用逐任务配置。

预算对等。将 GPT-5.2 上种子 42 的逐任务 GA 预算 $5\times$ 和 $10\times$ ，通过将最大进化次数从 4 提高到 20 和 40，将令牌消耗提高到每个种子 5.6M 和 11.4M，但留出 CO-Bench 测试仪从 0.502（该种子在 0.527 ± 0.033 三种子均值中的默认预算得分）提升至 0.615 和 0.628。逐迭代曲线在 0.4 至 0.6 区间内振荡，没有向 Meta n 在 17M 令牌下的 0.870 上升的趋势，因此我们没有进一步扩展对等。

自我编辑模式。在已发布接口的 Gemma 运行中，GA 在 `Symptom2Disease` 上触发了 4 次自我修改动作（两次成功的提示级调整，两次被验证器拒绝），在 `LawBench` 上触发了 6 次（逐步更雄心勃勃的多步提示流水线），在 `AlphaEvolve Math` 上触发了 1 次，在 `CO-Bench` 上触发了 1 次，而在 AlgoTune 的迭代中为零次，其中所有七个每次迭代的求解器快照字节完全相同。唯一的 AlphaEvolve 变异保留了种子的整数答案要求，尽管任务期望 Python 模块源代码，并且得分从 0.372 回退到 0.079。GA 会原地重写其单体求解器，并且必须选择这样做，而 Ω 总是发出注入，在未触及的内部求解器周围添加库函数和预处理字符串（附录 F）。